



# Linux 内核分析实验报告

## 题目 音乐厅门票订售系统

姓 名：田佳铭  
学 号：22920172204211  
院 系：信息学院人工智能系  
专 业：智能科学与技术  
年 级：2017 级  
指导教师：陈毅东

2020 年 6 月 15 日

## 目录

|                             |    |
|-----------------------------|----|
| 一、实验目的.....                 | 3  |
| 二、实验环境及工具.....              | 3  |
| 三、程序框架说明.....               | 3  |
| 3.1 程序整体架构.....             | 3  |
| 3.2 main.cc 说明.....         | 3  |
| 3.3 相关的类说明.....             | 4  |
| 3.4 其他头文件说明.....            | 5  |
| 四、具体实现方法.....               | 5  |
| 4.1 main.cc 用到的 IPC 机制..... | 5  |
| 4.1.1 ftok.....             | 5  |
| 4.1.2 fork.....             | 5  |
| 4.1.3 共享内存区.....            | 5  |
| 4.1.4 信号量.....              | 6  |
| 4.1.5 父进程等待所有子进程退出.....     | 6  |
| 4.2 座位状态与交易约束.....          | 6  |
| 4.2.1 座位状态.....             | 6  |
| 4.2.2 交易约束.....             | 6  |
| 4.3 一些类的成员函数实现思路.....       | 7  |
| 4.3.1 Aseat 类.....          | 7  |
| 4.3.2 Lseat 类.....          | 7  |
| 4.3.3 Hall 类.....           | 8  |
| 4.3.4 auto_work 函数.....     | 8  |
| 4.3.5 子进程处理过程.....          | 9  |
| 4.4 一些辅助函数实现思路.....         | 10 |
| 4.4.1 座位号与偏移转换函数.....       | 10 |
| 4.4.2 各种类型转换函数.....         | 10 |
| 4.5 测试.....                 | 10 |
| 4.5.1 测试安排.....             | 10 |

|                 |    |
|-----------------|----|
| 4.5.2 测试说明..... | 11 |
| 4.6 代码整体评估..... | 13 |
| 五、实验体会.....     | 13 |

## 一、实验目的

利用 Unix/Linux 的 IPC 机制仿真一个音乐厅门票订售系统。需要用到 Unix/Linux 的 IPC 机制。

关于实验的具体说明详见：2017 级《Linux 内核分析》课程项目描述与要求。

## 二、实验环境及工具

VMware, ubuntu 虚拟机, 版本: ubuntu16.04, 32 位。编程语言: C++。

## 三、程序框架说明

### 3.1 程序整体架构

"Hall\_Ticketing\_System"文件夹中, main.cc 是主程序, 在此目录下输入 make 命令, 则可以得到可执行文件 main, 直接输入"./main"即可执行。输入 make clean 则可以消除此可执行文件。

main.cc 使用到的所有头文件都在"include"中, 因为文件数目不多, 就没有分的更细。main.cc 的输入文件是"input"文件夹中的"input.txt"。

此外, 在"alternate"文件夹中还放有两个历史文件(备用), 都是可以运行的, "lab"文件夹使用方法和 Hall\_Ticketing\_System 相同; "lab2"文件夹需要输入: "g++ main.cc -o main"进行编译, 两者的实现和 Hall\_Ticketing\_System 略有不同。

### 3.2 main.cc 说明

main.cc 是主函数, 实现的整体框架为:

- (1) 判读输入文件是否存在, 如果不存在则退出
- (2) 从输入文件中读取所有的信息并对信息进行基本的判断
- (3) 创建共享内存区

- (4) 创建信号量
- (5) 在共享内存区中初始化音乐厅座位
- (6) 生成所有的代理（进程扇方式）
- (7) 父进程等待所有子进程，当所有子进程都退出，转到（9）
- (8) 子进程从自己的命令集合中取出一条命令，调用 Hall 类的 `auto_work` 函数进行相应的动作，子进程重复（8）直到命令执行完，子进程退出
- (9) 父进程打印所有座位信息

### 3.3 相关的类说明

本程序使用到了三个类：`Aseat`（对应 `Aseat.h` 和 `Aseat.cc`）、`Lseat`（对应 `Lseat.h` 和 `Lseat.cc`）和 `Hall`（对应 `Hseat.h` 和 `Hseat.cc`）。

`Aseat` 是描述单个座位的类，包括座位的状态（空闲、被预订和被购买）、座位的拥有者（仅座位的状态为被预订和被购买时有效）和预约座位的剩余时间（仅座位的状态为被预订时有效）。

`Lseat` 是描述一排的座位的类，包括此排座位数、此排空闲座位数和相应数量的 `Aseat` 类对象数组。

`Hall` 是描述整个音乐厅座位的类，包括座位排数、每排座位数、音乐厅空闲座位数、座位可预约时间和相应数量的 `Lseat` 类对象数组。

由此可见，这三个类的层次关系是很明显的，在程序中，我们只需要创建一个 `Hall` 类的对象，就可以描述整个音乐厅，而调用函数的一般流程也是从 `Hall` 类开始，去调用 `Lseat` 类的成员函数，而 `Lseat` 类的成员函数就会去调用 `Aseat` 类的成员函数。

例如，如果我们想要打印整个音乐厅座位的预/售情况，就需要调用 `Hall` 类的打印函数，而 `Hall` 类的打印函数通过调用其中每个 `Lseat` 对象的打印函数来实现，`Lseat` 类的打印函数也需要调用其中每个 `Aseat` 对象的打印函数来实现。

为了保证这个过程能够顺利进行，大部分函数的返回值都是 `bool` 值，方便更高层次的对象来知道低层次的对象成员函数调用是否成功。

为了简化主函数，在 `Hall` 类中还有一个 `auto_work` 函数，它可以根据输入的交易描述来自动选择需要执行的函数。

这三个类的具体函数调用关系比较复杂，在第四部分会通过一些函数简要说明，具体实现请参考代码。

### 3.4 其他头文件说明

`get_mess.h` 存放获取输入信息的函数，主程序通过调用其中的 `get_mess` 函数来获取所有输入函数中的信息。

`mysem.h` 存放有关信号量的结构和函数，包括 `union semun` 的声明、信号量中的 `Wait` 和 `Signal` 函数的实现。

## 四、具体实现方法

因为程序本身内容比较多，不可能全部描述，这里只选择一些比较重要的具体实现方法来讲。具体实现参考代码。

### 4.1 main.cc 用到的 IPC 机制

`main.cc` 主要用到以下一些 IPC 机制来进行进程间的同步：

#### 4.1.1 ftok

用以生成创建共享内存和信号量的键值。因为实验的时候经常出现申请失败的情况，这里采用遍历的方式，直到找到合适的键值。

#### 4.1.2 fork

用以产生子进程。因为我们需要的是一个进程扇，所以在创建的循环中让子进程直接退出此循环。

#### 4.1.3 共享内存区

我们需要一个共享内存区来存放一个 `Hall` 类的对象。这个对象在创建子进

程之前由父进程初始化。在子进程中，对 Hall 类对象的每次操作都认为是原子操作（将在后面说明）。最后，父进程调用 Hall 类对象的打印函数。在程序结束前，要销毁共享内存区。

#### 4.1.4 信号量

使用 System V 信号灯来实现信号量，因为我们认为子进程对于 Hall 类对象的每次操作都是原子操作，因此，我们只需要一个初始值为 1 的信号灯即可。通过 semctl 来实现 Wait 和 Signal 操作。在父进程结束前需要销毁信号灯。

#### 4.1.5 父进程等待所有子进程退出

父进程需要等到所有子进程都结束后才可以打印音乐厅信息和进行最后的一些处理（销毁共享内存区、信号量等）。

使用 `signal(SIGCHLD,SIG_IGN);`和 `wait` 函数来实现。

### 4.2 座位状态与交易约束

#### 4.2.1 座位状态

座位一共有三种状态：空闲、被预订和被购买。这三种状态的转换关系是：

- （1）当座位空闲时，它可以被预订或者被购买（不支持预订自己已经预订的座位）
- （2）当座位被预订时，仅预订者可以进行取消预订和购买
- （3）当座位超过预约期，则自动变为空闲状态

#### 4.2.2 交易约束

有关于交易，我们有如下的规定：

- （1）对于指定具体范围的购买和预订交易，只有当所选中的座位（或者座位集合）全部处于可以被购买或预订状态时，才可以进行操作。
- （2）对于没有指定具体范围的购买和预订交易，只有当整个音乐厅的座位足够进行操作时才可以进行操作。

(3) 对于没有指定具体范围的购买操作，优先购买已经预约过的座位。

(4) 在以上规则的约束的情况下，购买和预订都是遵循优先选择小号的原则（即从 A 排第 1 个座位，以遍历排优先的顺序一直遍历到最后一排最后一个座位）

(5) 对于取消预订交易，对选中的座位中可以被取消预订的座位进行操作，不一定需要选中的座位（或者座位集合）全部处于可以取消预订的状态。

(6) 在合法的交易进行后再进行时间流逝的操作，并且更新 Hall 对象的状态，对于不合法的交易，则默认不花时间。

### 4.3 一些类的成员函数实现思路

为了方便叙述，我们先将有效的交易类型进行命名。

对于"show all 客户名"交易，我们认为是 S 交易，对于"reserve"交易，我们认为是 R 类交易；对于"ticket"交易，我们认为是 T 类交易；对于"cancel"交易，我们认为是 C 类交易。

对于座位描述，认为"A 2"、"A 4 7"、"A"、"A 2r"、"A 2c"、"6"分别为 1-6 类交易，这样，一个交易就可以描述为类似 R-1 类交易。

#### 4.3.1 Aseat 类

Aseat 类需要提供一些基本的功能函数，以便上层调用。对弈一些简单的函数，例如状态更新、打印函数、获取变量值的函数这里就不过多介绍（包括后面两个类），实现起来比较简单。

比较重要的函数是对单个座位的交易函数。交易原则参考 4.2，具体实现都是先判定客户是否对此座位有交易权限（不需要进行姓名合法判定等操作，这些操作均在上层完成），如果符合权限，则进行座位信息修改并返回 true，否则返回 false。

#### 4.3.2 Lseat 类

可以看到，大多数交易描述都和行号有关，所以大部分交易都要在 Lseat 类中得以实现。

其中，直接和行有关的操作是 1、2、3、5 类交易。对于这些交易，首先判断此交易是否是可执行的（某些座位是否没有操作权限、是否越界等），如果不可执行，则打印错误信息并返回 `false`，否则调用相关的 `Aseat` 中的函数进行执行，执行成功后则返回 `true`。（既然已经判定可以执行，则一般都会成功）

具体的实现逻辑需要参照 4.2。

### 4.3.3 Hall 类

在 `Hall` 类中，我们需要解决剩下的一些交易的实现，主要是 4 类和 6 类，4 类交易的实现比较简单，只需要调用数排的 3 类交易实现函数即可，6 类的实现则需要实现得到客户可以操作的座位数（对于 R 类和 T 类交易），在通过遍历排的方式来实现。这两个函数的实现中也需要进行一定的边界判断等，并且，也需要返回一个 `bool` 值。

### 4.3.4 auto\_work 函数

`auto_work` 函数可以说是 `Hall` 类中的核心函数（甚至 `Hall` 类中可以只给出这一个接口函数），它的目的就是根据交易字符串来自动选择要执行的函数。

它的原型为：`int auto_work(string command,vector<int> t_list)`；它接收两个参数，一个是 `string` 类型的 `command`，即一条交易信息，另一个是一个 `int` 动态数组，动态数组中至少包含四个值，即每个代理的 `reserve_time`、`ticket_time`、`cancel_time`、`check_time`。如果命令是合法的，则会返回执行此交易所需要的时间，否则，返回 0。所有的交易信息都会在此函数中打印出来。

它的主要工作流程是：

- （1）判定输入的 `t_list` 是否合法，不合法则打印错误信息并返回 0；
- （2）将 `command` 变量按空格拆分成一个 `string`（动态）数组 `scom`；
- （3）对此 `scom` 的长度进行判定，如果小于 3 或者大于 5 都是无效命令，打印错误信息并返回 0；
- （4）对 `scom` 的最后一个元素进行判定，如果是非法姓名，则打印错误信息并返回 0；
- （5）对 `scom` 第一个元素进行判定，如果此交易不是 S、R、T、C 类交易（这

里还没有判定是否是这四种中其中一种的合法的交易)，则打印错误信息并返回 0；否则，进入不同的分支。

分支主要有两种情况，S 类和非 S 类。

对于 S 类的处理：检查 `scom` 的第二个元素是否是"all"，如果是，则调用相关函数（这时的调用一般不会失败，主要调用处理 S 类交易的函数和时间流逝函数）并返回此代理处理 S 交易所需要的时间；否则，打印错误信息并返回 0；

对于非 S 类的处理：

（1）根据剩余的参数判定是否是有效的 1-6 类交易，并且判定出是哪一类交易，如果不是，则打印错误信息并返回 0；

（2）调用相应的处理函数，如果处理函数的返回值是 `false`，则打印错误信息并返回 0；

（3）调用时间流逝函数，并且返回此代理处理此交易所需要的时间。

可见，整个函数中充满了各种各样的分支语句，用于识别出各种错误和选择合适的函数。

#### 4.3.5 子进程处理过程

子进程的处理过程为：

- （1）调用信号量的 `Wait` 函数；
- （2）和共享内存建立连接；
- （3）取下一条交易，调用 `auto_work` 函数，得到返回值；
- （4）如果返回值不为 0，则打印相关信息；
- （5）调用 `sleep` 函数；
- （6）和共享内存区断开连接；
- （7）调用信号量的 `Signal` 函数；
- （8）如果交易列表中还有交易，则转（1），否则，结束子进程。

这里我选择将每笔交易看成是一个原子操作。主要是出于如下的考虑：

（1）最基础的选择是为每个座位设置一个信号量，或者是考虑到大部分操作都是对于行来进行的，可以对每一行设置一个信号量。这会造成一些麻烦，例如，在每次交易执行时，我们都需要事先选好要操作的座位或者行，这样我们才

可以针对这些行和列进行“加锁”处理，这就要求我们在每次交易前的选择阶段都是原子操作（否则，加锁也会产生冲突），就需要额外的信号量来参与，更加麻烦的是，我们在每笔交易结束后都需要更新全部座位的信息（因为时间流逝了），这要求我们对所有的座位加锁，很可能在 Wait 处一直等待导致时间流逝的处理不及时...这样的问题很多，控制起来也极其复杂。

（2）将每笔交易看成是原子操作是合理和方便的。一方面，每笔交易都可能会修改所有座位的值，并且我们不希望在交易执行过程中突然失败，所以对于每笔交易我们最好是在一开始就能确定的直到它是否会成功，就不会出现中途出错的情况。另一方面，我们只需要一个信号量就可以实现这种互斥操作。

## 4.4 一些辅助函数实现思路

### 4.4.1 座位号与偏移转换函数

我们的座位号编码方式是 A、B，但是在程序中，我们需要使用偏移，在输出的时候，我们又需要转换为座位号，所以我们需要做座位号和偏移的转换。考虑到座位号可以粗略的看成是一个 26 进制数（实际上有些差别），座位号转偏移可以通过简单的遍历方式来实现。

对于偏移转座位号，我们也可以通过不断除以 26 的方式来获取。

### 4.4.2 各种类型转换函数

在程序过程中，有很多地方需要做类型转换，例如 string 和 int 的相互转换（原始交易是存储在 string 中的）、将 string 按空格拆分等，实现这些功能的函数很多，有一些是可以自己写的，另一些可以调用一些类型转换函数，例如 atoi 等，还有一些可以借助各种 io 类来进行控制。

## 4.5 测试

### 4.5.1 测试安排

在 input.txt 文件中，有一个我测试时的实例，这个实例是这样安排的：

（1）代理 1 和代理 2 想方设法来制造冲突。例如代理 1 的第一条交易是 AA

预订 A 排第 2 个座位，代理 2 的第一条交易是 BB 购买 A 排第 2 个座位，由于他们都是第一条交易，索引有很大可能会发生冲突。

(2) 代理 3 的作用是迫使时间流逝和不断检测 S 交易的处理情况。在最后，也进行了一些其他测试。我们给代理 3 很长的查询时间，这样再查询的过程中，早期预订的那些座位就自动变为空闲了，这样就可以检测座位状态转换是否成功，并且代理 3 的交易比较多，最后的一些交易一般是代理 3 的，我们就可以用同一个客户来检验座位转换情况。

(3) 代理 4 的作用是对各种非法交易进行检测。

初始的 n、m、k、rvt 分别设置为 5、10（设小一些来构造冲突）、4、1（时间：分钟，需要转换为 s，这个值不要太大，否则要耗尽前面交易的预订保留时间很难）

虽然这个实例不能考虑到所有的情况（而且各交易执行顺序也有一定的随机性），但是也是可以作为参考的，为了过程更清楚，在调试过程中的每一步也都可以调用打印函数来观察座位的状态。

#### 4.5.2 测试说明

对于每笔成功的交易，交易清单格式如下：

```
The result of command[ticket A 2 BB]:  
The seat 2 is ticketed  
It costs agent2: 3s
```

图 4-1 交易清单示例 1

```
The result of command[reserve 8 CC]:  
LineA:  
The seat 1 is reserved  
The seat 3 is reserved  
The seat 4 is reserved  
The seat 5 is reserved  
The seat 6 is reserved  
The seat 7 is reserved  
The seat 8 is reserved  
The seat 9 is reserved  
It costs agent1: 3s
```

图 4-2 交易清单示例 2

成功的交易的清单的格式为：

- (1) 第一行：显示出原始交易信息
- (2) 之后的若干行：显示交易清单
- (3) 最后一行：显示这是哪个代理做的，花了多少时间

根据交易的不同，清单内容差别比较大，就不再一一展示。

对于失败的交易，主要分为两类：交易格式错误和冲突。失败的交易的清单的格式为：

(1) 第一行：显示出原始交易信息

(2) 之后的若干行（通常是 1 行）：显示错误原因（默认错误时不需要花费时间）

交易格式冲突示例：

```
The result of command[reserve A 11 AA]:  
command out of range
```

图 4-3 越界错误（不存着这个座位）

```
The result of command[show all 1AA]:  
Invalid command, invalid name
```

图 4-4 非法姓名（以数字开头）

```
The result of command[ticket AA AA AA AA AA]:  
Invalid command, too many item
```

图 4-5 其他格式错误

对于座位被占用，我们可以看到：

```
The result of command[ticket A 2 BB]:  
The seat 2 is ticketed  
It costs agent2: 3s  
  
The result of command[reserve A 2 AA]:  
the seat 2 is occupied
```

图 4-6 座位占用

因为之前 BB 已经买了这个座位，所以 AA 就不能预订了（相反，如果 AA 先执行，则 BB 就无法购买）。

在所有子进程结束后，会打印所有座位的信息（这里用的是一个 3 排、每排 5 个座位的例子，结构会稍微清楚一点）：

```
Millsion completed, current sutuation:  
LineA: [seat1:[sold,host:DD],seat2:[free],seat3:[sold,host:DD],seat4:[ordered,host:CC, left time:46s],seat5:[ordered,host:CC, left time:46s]]  
LineB: [seat1:[sold,host:CC],seat2:[sold,host:CC],seat3:[sold,host:CC],seat4:[sold,host:CC],seat5:[sold,host:CC]]  
LineC: [seat1:[ordered,host:CC, left time:46s],seat2:[ordered,host:DD, left time:51s],seat3:[ordered,host:DD, left time:51s],seat4:[ordered,host:DD, left time:51s],seat5:[free]]
```

图 4-7 座位信息打印

可见，对于每一个座位，有三种情况：

(1) 如果座位空闲（蓝箭头所指），则显示"free"。

(2) 如果座位被预订（红线），则显示"ordered"，并且显示预订者和预订的剩余时间。

(3) 如果座位被预订（黄线），则显示"sold"，并且显示购买者。

## 4.6 代码整体评估

整体来说，我认为这个程序不算特别复杂，因为采用了三个类，每一层实现起来也比较清楚，因为采用三个类来实现音乐厅的设置，所以申请共享内存区的时候只需要申请一个 Hall 类的对象即可，并且控制起来比较简单。

不过程序也存在一些缺点，例如 `auto_work` 的实现代码过于长，可以想办法继续拆解。或者，因为预定、购买、取消预定的代码是比较相似的（主要是判断条件不同），所以可以考虑将部分代码进行进一步的合并。又例如，在程序的很多地方使用了一些 `io` 流来做类型转换，所以整体来说实现效率有所降低。

一个比较严重的问题是，代码不够灵活。在 Hall 类中，`Lseat` 类对象是定长数组，在 `Lseat` 类中，`Aseat` 类对象也是定长数组。这是因为在测试的时候，使用指针很可能会导致共享内存失败，初步推测是因为对于不同的地址空间，处理同一个指针值时，所映射到的物理空间是不一样的，所以就删除了原先的采用用指针来实现的思路（也许可以尝试通过深复制来尝试解决这个问题，但实际调试起来也出现了一些问题），这导致程序会出现一些无用的空间（有做越界判断，所以不用担心数组越界的问题）。

## 五、实验体会

经过几个学期的学习，对于 Linux 编程也有一些自己的感想，虽然对于 Linux 还谈不上是完全熟悉（例如在写 Makefile 的时候也只是写了几条基本的语句），但对于 Linux 中的很多机制还是可以应用的。

这个实验的前期设计花费了我不少时间，我开始的思路是试图给每一个座位都设置一个信号量，在理论分析阶段就排除了这个想法，我也咨询了学长，去年 Linux 实验的飞机票订售系统好像有要求每笔交易都是原子操作，于是就放心的

做下去了，因为采用了三个类来表示音乐厅的座位，整体结构比较清楚，大部分函数也比较短，但是要搞清函数之间的关系也是花了一些功夫的。有些东西也经常搞得我比较晕，偏移、座位号和排号之间经常混用，导致出现过很多错误。整个程序比较复杂，虽然我也做过不少测试，但我也不敢说当前的程序没有 bug。

实验中还是有很多闹心的地方，一开始的时候我是直接指定键值，导致共享内存申请经常失败，最麻烦的是我发现共享内存中的指针变量存在的问题（我是通过 string 类来进行测试，发现这个问题的），这导致我不得不大面积修改程序。不过，就实验目的来说，我还是学会了 IPC 机制的简单应用，程序测试起来效果也还不错（虽然输出格式看起来有点丑，但我也真的不太会设计）。

整个系统做下来，我还是学到了很多东西的，包括这次我也尝试着把代码写成开源代码常见的格式，我觉得，这个实验对我的提升还是全方面的，写出来也比较有成就感。